

销；上层应用必须使用多内核提供的进程间通信接口才能进行交互，需要移植现有应用才能适应多内核架构；绝对性能方面并不一定存在优势。

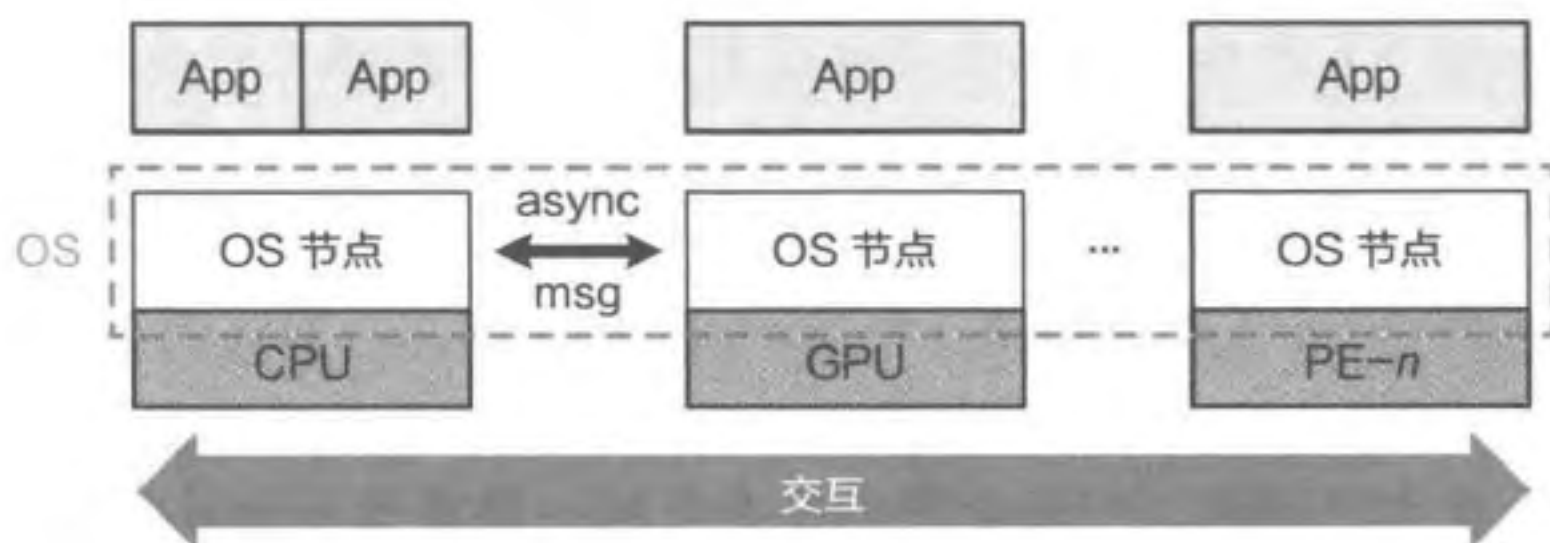


图 2.8 多内核操作系统的架构

混合内核架构。由于设计需求的多样化，现实中的操作系统往往融合了多种架构的设计思想。图 2.9 展示了当前微内核、宏内核、外核、多内核等架构的演进及其在现实操作系统中的体现。例如，苹果操作系统的内核 XNU 是 Mach 微内核与 BSD UNIX 的混合体；Windows NT 操作系统的内核也采用了微内核设计思想，但为了实现更低的操作时延，选择将一部分系统服务（特别是与图形界面相关的服务）运行在内核态；Linux 虽然是宏内核架构，但近期也开始支持用户态的驱动框架（如 UIO^①与 VFIO^②等）。

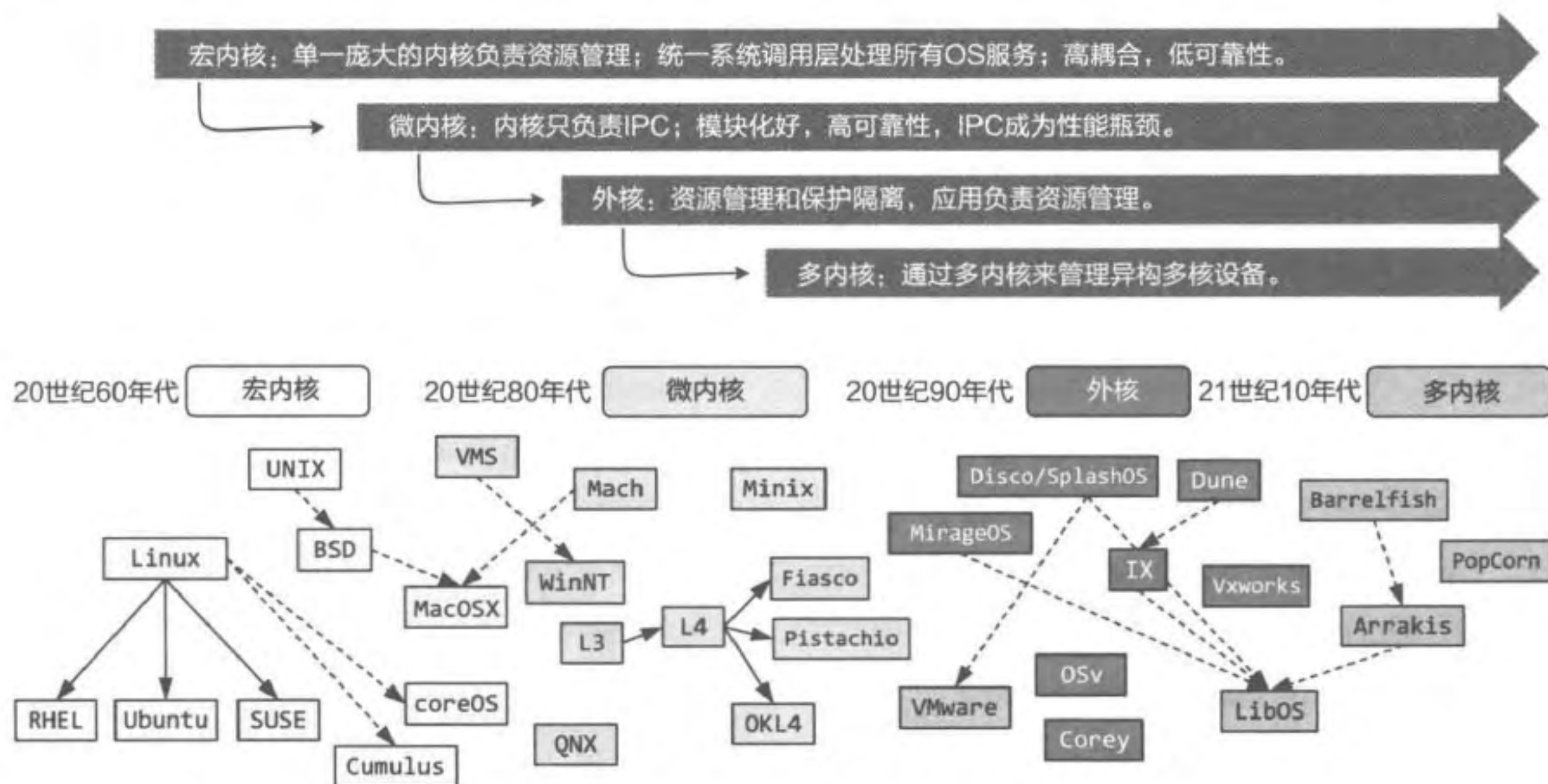


图 2.9 操作系统的架构组合及其演进

① 参见 <https://www.kernel.org/doc/html/v4.18/driver-api/uio-howto.html>。

② 参见 <https://www.kernel.org/doc/html/latest/driver-api/vfio.html>。

小知识：Windows 也采用了微内核的设计思想吗？

是的。事实上，微软研究院的创始人 Richard Rashid 就是第一代微内核 Mach 的设计者。从 Windows NT 开始，微软将与处理器和体系结构相关的功能放入一个很小的微内核，而将其他功能（如进程管理器、虚拟内存管理器、进程间通信管理器、I/O 管理等）以模块化的方式实现，称为 executive。与纯微内核不同的是，Windows NT 将微内核与 executive 链接到一个模块（称为 ntoskrnl.exe），共同运行在处理器的特权模式下。这意味着不同 executive 的函数间可以互相直接调用，具有很低的时延，但也意味着一旦某个函数出现问题，可能会影响整个系统，隔离性不如纯微内核。

2.4 操作系统框架结构

本节主要知识点

- ❑ 为什么说 Android 的应用框架与微内核架构相似？
- ❑ Android 成为最流行的移动终端操作系统，这与它的结构有哪些关系？

如 1.2 节所述，现代操作系统一般由操作系统内核和操作系统框架（包含系统服务与应用框架）构成，本节将简要介绍两个操作系统（Android 和 ROS）的框架。为了简化开发并提供可演进性与可维护性等，许多操作系统框架的结构设计借鉴了类似微内核架构的思想，将操作系统系统框架进行组件化与服务化，并提供进程间通信以支撑各个组件进行交互。

2.4.1 Android 系统框架

Android 是当前最流行的移动终端操作系统。截至 2019 年年底，Android 已经被部署到全球 74.13% 的智能终端设备上。Android 的设计目标之一是方便各个厂商适配，整体采用了更加商业友好的 Apache Software License。与 GPL（GNU Public License）不同的是，Apache Software License 不要求使用并修改源码的开发者重新开放源码，而只需要在每个修改的文件中保留 License 并表明所修改的部分。



图 2.10 Android 操作系统的架构

图 2.10 展示了 Android 操作系统的

架构。在 Linux 内核之上运行的 Android 系统框架包括如下几个主要组件。

- **Android 硬件抽象层 (Hardware Abstract Layer, HAL)**。Android 在 Linux 内核上提供硬件抽象层的主要原因有两个：第一，Linux 设备驱动的接口与内核版本之间存在耦合，这会阻碍 Android 系统框架的独立演进与升级；第二，Linux 内核采用 GPLv2 开源协议，要求运行在同一个地址空间的设备驱动必须开放源码，这可能会导致一些硬件的实现细节也被公开，厂商对此存在顾虑。因此，Android 提供硬件抽象层，封装一些硬件实现的细节，从而更好地实现 Linux 内核与 Android 系统框架的解耦，并通过提供用户态驱动框架，使得设备厂商不需要开放源码就能为 Android 操作系统提供设备驱动，从而促进更多的设备厂商加入 Android 生态。
- **Android 库 (Library)**。Android 库一方面提供了一些方便 Android 应用开发的自定义库，另一方面也重新实现了一些标准库（如 glibc 等），从而规避了一些开源协议的限制。
- **Android 运行时 (Android Runtime, ART)**。Android 应用的主要开发语言是 Java，因此需要一个运行时环境将应用从字节码转化为可执行代码。早期的 Android 采用类似 Java 虚拟机的形式（称为 Dalvik 虚拟机），通过解释执行与 JIT (Just-in-Time) 编译的方式运行，这不可避免地导致了性能开销与功耗增加。Android 自 5.0 后引入 ART，通过 Ahead-Of-Time (AOT) 预先编译的方式，将 Java 代码预编译为二进制可执行代码，从而避免了运行时的编译开销。
- **Android 应用框架 (Application Framework)**。Android 应用框架提供应用运行所需要的基础服务，包括服务管理 (Service Manager)、活动管理 (Activity Manager)、包管理 (Package Manager)、窗口管理 (Window Manager) 等。这些服务化的组件利用操作系统内核提供的资源抽象，为应用构建了一系列方便调用的系统服务。

服务化架构与 Binder IPC。Android 系统框架的设计整体应用了类似微内核架构的思想，将系统框架组件化与服务化，并通过 IPC 进行交互，称为 Binder IPC。图 2.11 展示了 Android 应用框架的服务化架构：服务管理组件负责各个服务组件的注册与管理，应用通过服务管理组件完成服务发现 (Service Discovery)，然后通过 Binder IPC 调用各个服务组件请求服务。通过类似微内核架构的服务化框架，Android 将各种不同的服务进行解耦，

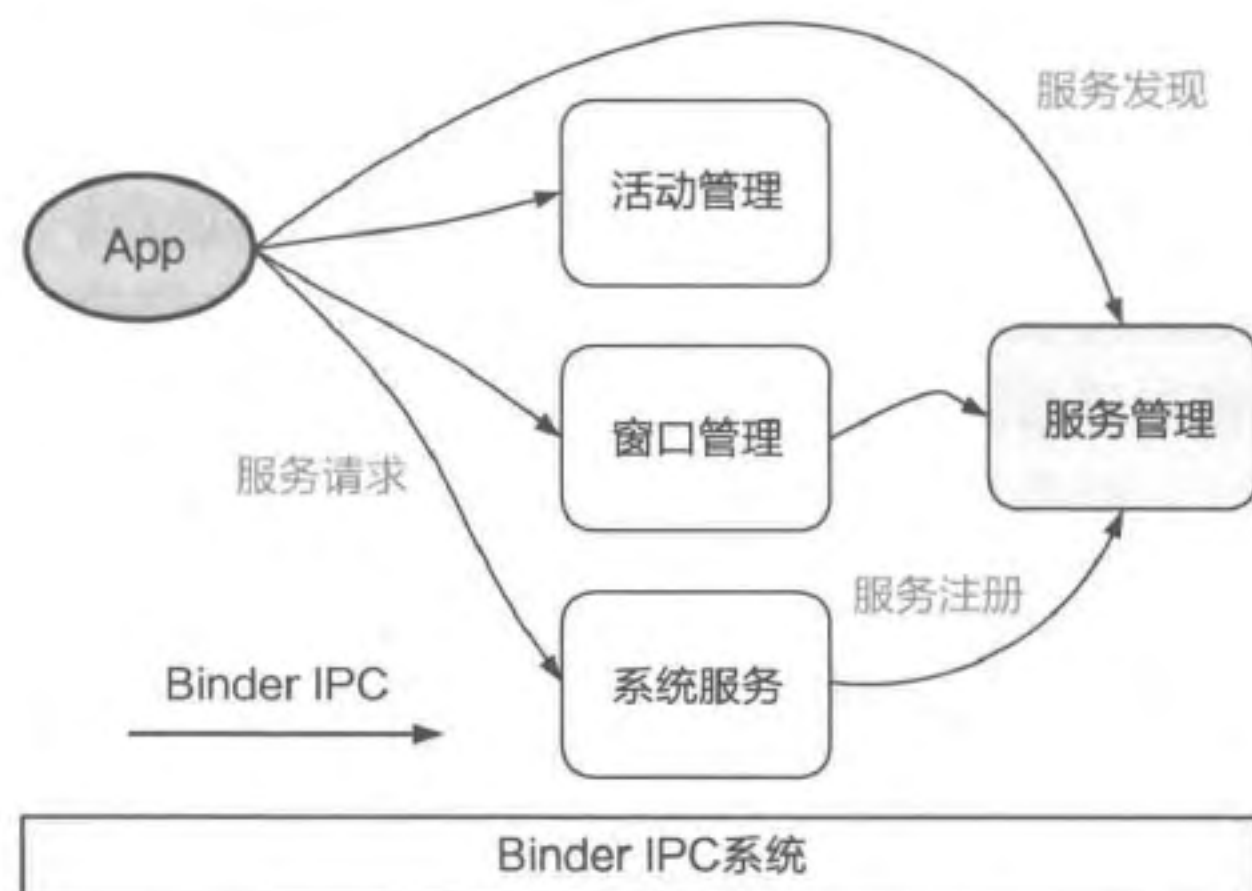


图 2.11 Android 应用框架的服务化架构

显著提升了系统的可扩展性与可维护性。

2.4.2 ROS 系统框架

ROS 是 Robot Operating System 的缩写，即机器人操作系统。虽然名字中包含操作系统，但 ROS 实际上是一个面向机器人硬件场景的系统框架，可运行在 Linux 内核以及其他兼容 POSIX 接口的操作系统内核之上。作为操作系统的系统框架，ROS 提供了一系列面向机器人场景应用的系统级服务，实现了底层复杂的异构硬件资源的抽象，并提供了应用之间和应用与服务之间交互的 IPC 机制。

图 2.12 展示了 ROS 的层次架构。简而言之，ROS 可以分为基础设施层、通信层和应用层。其中，基础设施层包括 ROS 中的 Python 和 C++ 等运行环境、ROS 文件系统、ROS 包管理机制等。在基础设施层之上运行 ROS 的通信层，提供基础的 ROS IPC 机制以及 ROS 的数据分发服务（Data Distribution Service）等。ROS 的应用层是由一个个节点所构成的计算图，这些节点之间可以使用 ROS 的通信层进行直接调用，或者通过对一些话题（Topic）发布消息或订阅消息以进行间接通信。

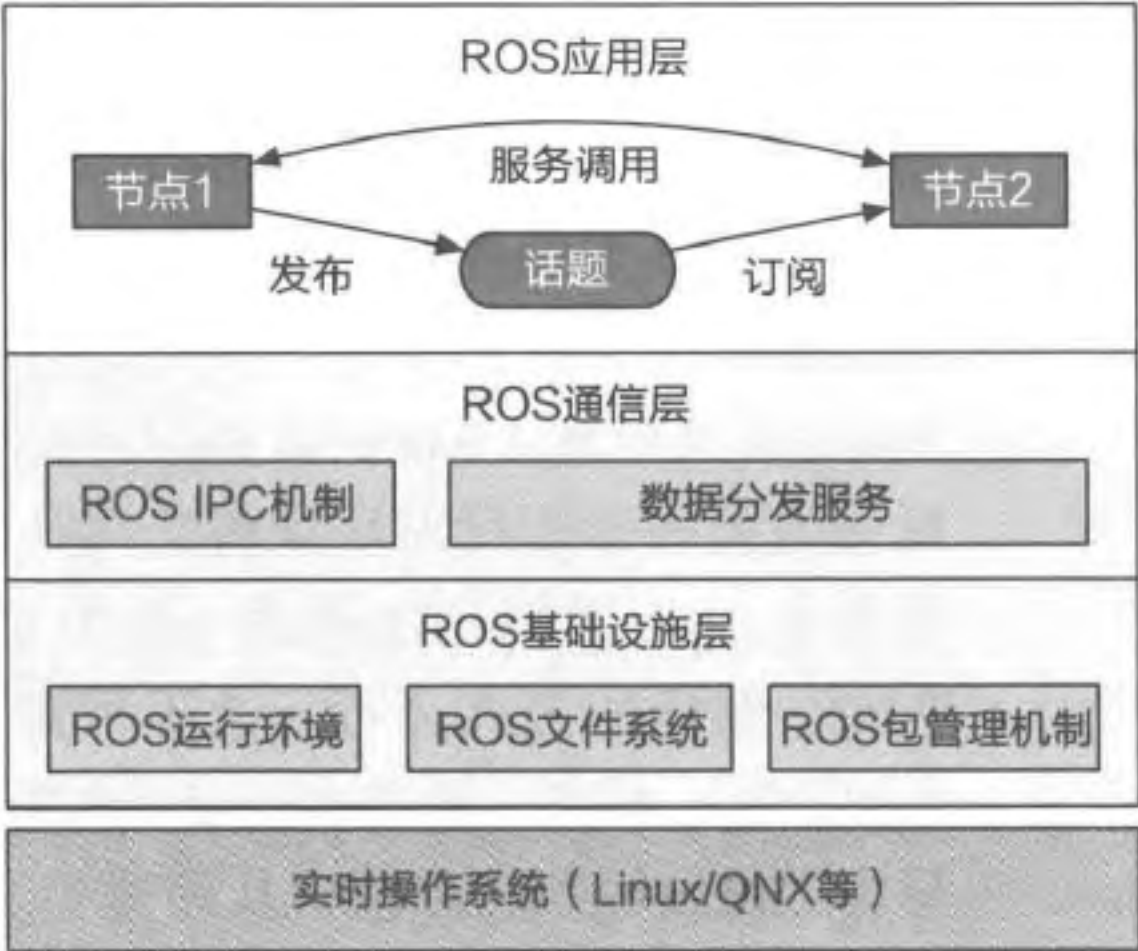


图 2.12 ROS 的层次架构

ROS 节点与计算图。ROS 采用计算图的形式来组织各种服务。每个服务构成一个节点，每个节点执行一个具体的计算处理，包括机器人中所需的感知、融合、规划与控制等。运行在 ROS 上的应用可以被表示成一个“图”的架构，不同的节点表示不同的应用（它们既可以是接收信息者，也可以是信息发布者）。ROS 上常见的应用包括：接收、发布、处理来自传感器（Sensor）的数据，以及对处理过程进行控制的节点与计划节点等。

与微内核的设计思想类似，ROS 同样将其系统框架分解为多个服务，不同的服务之间通过 ROS 提供的 IPC 进行通信。这种解耦式的架构使得 ROS 易于部署，甚至可以部署在分布式系统上。

服务与动作。ROS 每个组件的设计是以服务为中心的：服务接收一个请求，执行请求，然后回复结果。一个节点可以向另一个提供服务的节点发送请求，然后等待结果。服务的模型是一次性的同步通信，即当服务的请求和回复完成后，连接会被断开，进行下一次交互时必须再次建立连接。节点还可以提供动作（Action）。动作和服务类似，主要的区别在于动作的假设是服务端处理请求的时间会很长，并且可能需要在处理的过程中就获取一些中途的返回值。在实际的使用中，一般通过动作来让机器人执行相对复杂的任务。

话题。话题是 ROS 中比较新的处理方式，是一类单向的异步通信机制。话题机制将通信的双方称为发布者（Publisher）和订阅者（Subscriber）。发布者节点首先需要向主节点（一个特殊的全局节点）注册一个话题，之后该发布者会以消息的形式发布关于该话题的新内容。希望接收该话题的订阅者节点则通过主节点获得对应话题发布者的消息，基于这个消息，订阅者节点可以直接连接到发布者节点来接收最新的话题消息。在话题模式下，发布者和订阅者是多对多的关系，即单个节点可以成为多个话题的发布者，也可以成为多个话题的订阅者。

数据分发服务。ROS 2.0 引入了数据分发服务（Data Distribution Service, DDS）以支持实时性。从架构的角度，DDS 可以被看作一个系统中间件，处于 ROS 应用层和 ROS 基础设施层之间。DDS 提供了上面介绍的 ROS 基本通信接口和语义，如话题方式的发布者和订阅者模型或服务类型的 IPC 等。之所以将 DDS 分离出来，是为了加速对性能有重要影响的通信。不同的硬件厂商可以针对自己的机器人环境实现 DDS 的中间件，只要最终这些 DDS 的实现能够满足对应的协议接口即可。这些 DDS 是基于特定硬件实现的，因此可以进行可定制的优化。这样一来，ROS 的应用只需要考虑 DDS 的抽象，而不需要考虑底层硬件的细节，便可以使用当前环境下最高效的通信方式。

有了 DDS 抽象后，ROS 2.0 可以抛弃 ROS 中基于主节点的服务发现方式。这依赖于 DDS 提供的一套分布式发现系统（Distributed Discovery System），从而避免在主节点上出现性能瓶颈。ROS 需要通过 DDS 提供的 API 来获取，如当前存活的节点、当前的话题、当前注册的动作等信息。对于这些信息的查询都被 ROS 进行了抽象，对 ROS 上的应用而言其实是透明的。

DDS 内部可以利用共享内存的方式来传递本地数据。在 ROS 1.0 中，从系统底层来看，一次传输消息是先序列化数据，然后通过网络（TCP）把数据传递出去，即使是本地数据交互也会通过 localhost 等接口传输。而在 ROS 2.0 中，ROS 将数据交接给 DDS 层。具体的 DDS 实现可以使用共享内存等方式在不同节点之间共享数据，从而极大地提升传输性能。

计算图架构与 ROS IPC。ROS 系统框架的设计将计算任务组件化与服务化，形成独立的 ROS 节点，各个节点通过 ROS IPC（如话题、服务、动作等方式）进行通信。DDS 的引入能够在支持多种通信方式的情况下，尽可能地优化底层通信性能。这些设计充分体现了机制与策略分离的原则。类微内核设计的解耦式架构，使得 ROS 系统框架及其应用易于部署在硬件环境差异极大的机器人平台上，并且显著提升了单个节点的可复用能力。

2.5 操作系统设计：Worse is better?

小背景：PC loser-ing 问题

PC loser-ing 问题最早出现在 MIT ITS 操作系统的一个场景中^[15]；